Finite Differences Pricer

1 Overview

The finite differences method solves the Black-Scholes PDE directly by discretising the stock price and time dimensions onto a grid. Unlike the binomial tree (which builds an explicit tree of stock prices) or Monte Carlo (which simulates paths), finite differences works backwards through a grid of option values — making it well-suited to American options where early exercise must be enforced at every point in (S, t) space.

2 The Black-Scholes PDE

Under the risk-neutral measure, the option value $V(S, t)$ satisfies the Black-Scholes PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + (r - q)S\frac{\partial V}{\partial S} - rV = 0$$

It is more natural to work in **time-to-maturity** $\tau = T - t$ (so $\tau = 0$ is expiry and $\tau = T$ is today). Reversing time turns the PDE into a forward equation:

$$\frac{\partial V}{\partial \tau} = \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + (r - q)S\frac{\partial V}{\partial S} - rV$$

Starting from the known terminal condition at $\tau = 0$ (the option payoff), we march forward in τ — stepping backwards through calendar time — until we reach $\tau = T$ (today's option price).

3 Grid Setup

A uniform grid is placed over the region $[0, S_{\max}] \times [0, T]$:

- $S_j = j \cdot \Delta S$ for $j = 1, \ldots, M$ (interior nodes only; $S_0 = 0$ and $S_{M+1} = S_{\max}$ are boundary nodes)
- $\tau^n = n \cdot \Delta\tau$ for $n = 0, 1, \ldots, N$

The grid spacing is $\Delta S = S_{\max}/(M + 1)$ and the time step is $\Delta\tau = T/N$. $S_{\max}$ is chosen as a multiple of $\max(S_0, K)$ — large enough that the boundary conditions there are not binding.

4 Crank-Nicolson Scheme

The simplest stable scheme is **explicit Euler** (compute V^{n+1} purely from V^n), but it requires a very small time step for stability. The **fully implicit** scheme is unconditionally stable but only first-order accurate in time.

Crank-Nicolson (CN) averages the two: it is unconditionally stable and **second-order accurate in both** ΔS **and** $\Delta\tau$, making it the standard choice in derivatives pricing.

Define the spatial operator $\mathcal{L}$ acting on V at interior node j:

$$\mathcal{L}[V]_j = a_j V_{j-1} + b_j V_j + c_j V_{j+1}$$

where the coefficients follow from the discretised PDE (central differences for both first and second derivatives):

$$\alpha_j = \frac{\sigma^2 S_j^2}{2\,\Delta S^2}, \qquad \beta_j = \frac{(r-q)\,S_j}{2\,\Delta S}$$

$$a_j = \alpha_j - \beta_j, \qquad b_j = -2\alpha_j - r, \qquad c_j = \alpha_j + \beta_j$$

The CN time step advances from τ^n to τ^{n+1} by averaging the explicit and implicit operators:

$$\frac{V^{n+1} - V^n}{\Delta\tau} = \frac{1}{2}\mathcal{L}[V^{n+1}] + \frac{1}{2}\mathcal{L}[V^n]$$

Rearranging separates knowns (V^n) from unknowns (V^{n+1}):

$$\underbrace{\left(I - \frac{\Delta\tau}{2}\mathcal{L}\right)}_{\text{LHS matrix } A} V^{n+1} = \underbrace{\left(I + \frac{\Delta\tau}{2}\mathcal{L}\right)}_{\text{RHS matrix } B} V^n + \text{BC}$$

5 Tridiagonal System

The matrix A has the structure:

$$A = \begin{pmatrix} 1 - \frac{\Delta\tau}{2}b_1 & -\frac{\Delta\tau}{2}c_1 & & \\ -\frac{\Delta\tau}{2}a_2 & 1 - \frac{\Delta\tau}{2}b_2 & -\frac{\Delta\tau}{2}c_2 & \\ & \ddots & \ddots & \ddots \end{pmatrix}$$

This tridiagonal system is solved at each time step using `scipy.linalg.solve_banded`, which exploits the banded structure for $\mathcal{O}(M)$ cost per step (rather than $\mathcal{O}(M^3)$ for a general solver).

The RHS matrix B is similarly tridiagonal, and the matrix-vector product BV^n is computed directly without forming B explicitly:

$$\text{rhs}_j = (1 + \tfrac{\Delta\tau}{2}b_j)\,V_j^n + \tfrac{\Delta\tau}{2}a_j\,V_{j-1}^n + \tfrac{\Delta\tau}{2}c_j\,V_{j+1}^n$$

5.1 Boundary Corrections

At the first and last interior nodes, the stencil reaches outside the grid. These boundary contributions are moved to the right-hand side as known corrections. For Crank-Nicolson they involve boundary values at both the old and new time levels:

$$\text{rhs}_1 \mathrel{+}= \tfrac{\Delta\tau}{2}\,a_1\left(V_{\text{left}}^n + V_{\text{left}}^{n+1}\right)$$
$$\text{rhs}_M \mathrel{+}= \tfrac{\Delta\tau}{2}\,c_M\left(V_{\text{right}}^n + V_{\text{right}}^{n+1}\right)$$

6 Boundary Conditions

The boundary conditions have clear economic interpretations.

Call option: - $V(0,\tau) = 0$: a call on a worthless stock is worthless - $V(S_{\max},\tau) = S_{\max} - Ke^{-r\tau}$: deep in-the-money, the call behaves like a forward contract

European put: - $V(0,\tau) = Ke^{-r\tau}$: with certainty the put pays K at expiry, discounted to present value - $V(S_{\max},\tau) = 0$: far out-of-the-money, the put is worthless

American put: - $V(0,\tau) = K$: at $S = 0$ under GBM the stock stays at zero forever, so the holder exercises immediately and receives K (no discounting — the payoff is obtained now, not at expiry) - $V(S_{\max},\tau) = 0$: same as European

7 Early Exercise Constraint

For European options, the CN system fully describes the price. For American options, we must additionally enforce the **free boundary condition**: the option value cannot fall below the intrinsic value at any node.

This is formally a **Linear Complementarity Problem (LCP)**:

$$V^{n+1} \geq h^{n+1}, \qquad AV^{n+1} - BV^n \geq 0, \qquad (V^{n+1} - h^{n+1}) \cdot (AV^{n+1} - BV^n) = 0$$

where h denotes the intrinsic value. The exact LCP can be solved via Projected SOR (PSOR). The implementation here uses the simpler **operator-splitting** approach: solve the CN system as if it were a European option, then project the result onto the feasibility constraint:

$$V^{n+1} \leftarrow \max(V^{n+1},\ h)$$

This is applied at every τ-step. The approach is second-order accurate for smooth free-boundary problems and produces prices within a few cents of PSOR for standard American options.

8 The `FiniteDifferencesPricer` Class

`FiniteDifferencesPricer` is a dataclass with three configuration fields:

Field	Default	Description
`n_s`	200	Number of interior S grid points
`n_t`	200	Number of time steps
`s_max_factor`	3.0	$S_{\max} =$ `s_max_factor` $\times \max(S_0, K)$

It accepts both `EuropeanOption` and `AmericanOption`. For European options the early exercise projection is skipped; the CN scheme alone recovers the Black-Scholes price.

```
from options_pricer import (
    AmericanOption, BlackScholesModel, ClosedFormPricer, EuropeanOption,
    FiniteDifferencesPricer, MarketData, OptionType, TreePricer,
)

md    = MarketData(spot=100.0, rate=0.05)
model = BlackScholesModel(vol=0.20)

eur_put = EuropeanOption(option_type=OptionType.PUT, strike=100.0, expiry=1.0)
am_put  = AmericanOption(option_type=OptionType.PUT, strike=100.0, expiry=1.0)

fd = FiniteDifferencesPricer()

# European: matches closed-form within a few cents
print(fd.price(eur_put, md, model))  # ~5.57
print(ClosedFormPricer.price(eur_put, md, model))  # 5.5735

# American: early exercise premium ~0.32
print(fd.price(am_put, md, model))   # ~5.89
```

8.1 Comparing to the Tree

Both the tree and FD pricer should agree closely for American puts. FD generally has smaller oscillations than CRR as the grid is refined.

```python
am_put = AmericanOption(option_type=OptionType.PUT, strike=100.0, expiry=1.0)

tree_price = TreePricer(n_steps=1000).price(am_put, md, model)
fd_price   = FiniteDifferencesPricer(n_s=200, n_t=200).price(am_put, md, model)

print(f"Tree (N=1000):  {tree_price:.4f}")
print(f"FD (200×200):   {fd_price:.4f}")
print(f"Difference:     {abs(fd_price - tree_price):.4f}")  # typically < 0.03
```

8.2 Effect of Grid Refinement

```python
for n in [50, 100, 200, 500]:
    p = FiniteDifferencesPricer(n_s=n, n_t=n).price(am_put, md, model)
    print(f"n_s=n_t={n:4d}:  {p:.5f}")
```